

Egor69 — Analyse de plateforme (A à Z)

Document généré pour le dépôt **peltiez** · Domaine canonique : <https://egor69.ca>

Portée : produit, architecture, données, monétisation, risques et pistes de pérennité.

1. Synthèse exécutive

Egor69 est une application web monopage (React 18, Vite 6) à surface fonctionnelle très large : marketplace circulaire, atlas et fiches vivantes, encyclopédie biblique (carte, graphe, scènes 3D), monde 3D (Verse), nombreux hubs thématiques, dashboards « radar », pricing Stripe, pages légales et de transparence, personnalisation « mon univers », PWA (service worker). Un noyau API Express (**sovereign core**) concentre Stripe, l'Atlas côté serveur et des métriques agrégées.

Une partie des données « métier » côté navigateur repose encore sur un **client local** (localStorage, préfixes `igor:`) — utile pour démo et prototypes — distinct des flux financiers et Atlas réellement câblés sur le serveur.

2. Identité et promesse publique

- **Site canonique** et SEO centralisés dans `src/lib/site.js` (`SITE_ORIGIN`, `SITE_TAGLINE`, `SUPPORT_EMAIL`).
- **Éthos** (`WORLD_ETHOS`) : souveraineté, transparence, pas de secrets dans le bundle, HTTPS en production, cadre commun pour tous les utilisateurs.
- **Porte d'accueil** et **décompte de lancement** pilotés par `experienceFlags.js` (dates symboliques documentées dans le code).

3. Architecture technique

Couche	Rôle principal
Front (Vite + React)	Router volumineux (~128 routes dans App.jsx), lazy loading des pages, providers (thème, auth « souverain » minimal, TanStack Query), Layout avec sidebar et navigation filtrée (universePreferences).
Build & qualité	npm run verify = ESLint + tsc + build Vite ; dédup React ; chunk react-vendor pour limiter les doubles instances de React en production.
API locale	server/index.js lit .env.server, monte createSovereignApp sur le port 8787 ; en dev, Vite proxifie /api vers ce port.
PWA	public/sw.js — stratégie de cache versionnée ; les assets /assets/ et fichiers .js ne doivent pas recevoir de fallback HTML (risque historique de corruption d'exécution).

4. Données côté client (prototype)

Le module `src/api/igorClient.js` (réexporté comme `base44`) fournit des entités (annonces, « consciousness director », etc.) stockées en **localStorage**, avec seeds minimales pour les écrans courants. Cela permet une démonstration riche sans backend métier complet sur chaque route.

Implication produit : distinguer clairement pour les utilisateurs et partenaires ce qui est synchronisé serveur (Stripe, Atlas, CRM) et ce qui est local / démo — argument clé pour la confiance et la monétisation B2B.

5. API souveraine (Express)

Fichier pivot : `server/sovereignApp.js`. Exemples de responsabilités :

- **Santé** : GET /api/health
- **Atlas** : compteur fiches vivantes, aperçu, fiche par scanId, conversion batch (secret optionnel IGOR_ATLAS_BATCH_SECRET)
- **Stripe** : sessions checkout, payment intents, webhooks, agrégats d'abonnements actifs
- **CRM** : capture de leads validée (schémas Zod)
- **Sécurité** : Helmet (sans CSP sur l'API JSON), CORS sur liste d'origines, rate limiting par famille de routes, corps JSON limités

6. Parcours utilisateur (aperçu)

- **Accueil** : mur radar, planète 3D (lazy), sections valeur, SEO (SEOMeta), notifications de proximité / compteur Atlas.
- **Hub Soutien** : route /soutien — regroupe connaissance (Atlas, pricing), communauté (abonnement, contact), territoires (fondations, contact structuré).
- **Monétisation** : /pricing (passes Stripe documentées), /abonnement (autre palier), emails support.
- **Connaissance immersive** : Atlas, Bible (multi-vues), Genome, Sentinelle, Authenticity, etc.
- **Loisirs / immersion** : Verse 3D, arène virtuelle, nombreux hubs (nature, spiritualité, jeu...).

7. Observabilité et erreurs

- **Sentry** (optionnel) via `VITE_SENTRY_DSN` — import dynamique pour ne pas alourdir le dev.
- **ErrorBoundary** avec message utilisateur en français.
- Les libellés **AggregateError** vus dans des outils tiers (ex. Sentry, dépendances) ne proviennent en général pas du code applicatif `src/` — souvent agrégation d'erreurs liées ou recherche dans `node_modules`.

8. Stratégie de revenus (rappel)

Freemium de la connaissance

Base gratuite (lecture / socle) ; passes et options avancées (radar, profondeur, packs produit) alignés sur Stripe — frontière à exprimer sans ambiguïté.

Micro-contributions & gouvernance

Abonnements, badges, soutien récurrent ; transparence sur l'usage des fonds renforce la crédibilité.

Services B2B / B2G

Tableaux de bord territoriaux, ateliers, exports — à traiter comme une offre contractuelle (DPA, SLA), distincte du parcours grand public.

9. Points forts pour la pérennité

1. Séparation nette entre **démo locale** et **noyau payant / Atlas serveur**.
2. Bonnes pratiques API déjà en place (CORS, rate limit, validation, pas de secrets front).
3. Pipeline `verify` reproductible avant déploiement.
4. Identité de marque et SEO centralisés.

10. Risques et recommandations pragmatiques

- **Double vérité données** : documenter ce qui est local vs serveur ; réduire la surprise pour les payeurs.
- **Surface produit** : prioriser tests, accessibilité et performance sur les parcours « jour J » (21 mai) avant d'étendre.
- **Documentation déploiement** : aligner les exemples d'URL (README serveur, variables d'environnement) sur le domaine canonique actuel.
- **Narration vs conformité** : isoler les écrans « ludiques / cosmiques » des offres institutionnelles sobres.

11. Glossaire rapide

Terme	Sens dans ce dépôt
Sovereign core	API Express Stripe + Atlas + CRM minimal
igorClient / base44	Couche entités locale (localStorage), nom historique conservé
Fiches vivantes	Contenus Atlas alimentés par scans / conversion batch
experienceFlags	Interrupteurs produit (porte d'accueil, décompte, Gros Câlin, etc.)

Document interne — Egor69 / peltiez. Source : docs/Egor69-Analyse-Plateforme.html. Pour régénérer le PDF : ouvrir ce HTML dans Chrome ou Edge puis **Fichier** → **Imprimer** → **Enregistrer au format PDF** (ou ligne de commande headless --print-to-pdf pointant vers l'URL file:/// du fichier HTML).